

Informática II

Informe de Responsabilidades

Proyecto Sweet Crush

Desafío 1 – Juego tipo arcade en C++

Integrantes

Andrés Felipe Sierra Fernández

Jesús David Guardiola

Repositorio:

github.com/Felipesierra2/Desafio1_informatica_2

21 de septiembre de 2026

Índice

1. Introducción	2
2. Criterio de reparto	2
3. Responsabilidades de Andrés Felipe Sierra Fernández	2
3.1. Archivo memoriatablero.cpp	2
3.2. Archivo tablero.cpp	3
4. Responsabilidades de Jesús David Guardiola	3
4.1. Archivo tablero.cpp	3
4.2. Archivo memoriatablero.cpp	4
4.3. Archivo main.cpp	4
5. Observaciones sobre el código	4
6. Conclusión	5

1. Introducción

Este documento describe la distribución de responsabilidades entre los dos integrantes del proyecto **Sweet Crush**, un juego tipo arcade desarrollado en C++ en el que el tablero se almacena en memoria dinámica y cada ficha ocupa **3 bits** concatenados dentro de una secuencia de bytes.

El proyecto se compone de tres archivos fuente con funciones (`main.cpp`, `memoriatablero.cpp` y `tablero.cpp`) y sus respectivos archivos de cabecera (`memoriatablero.h` y `tablero.h`), que solo contienen declaraciones. En total se identificaron **33 funciones**, distribuidas así:

Archivo	Funciones
<code>memoriatablero.cpp</code>	15
<code>tablero.cpp</code>	17
<code>main.cpp</code>	1
Total	33

2. Criterio de reparto

El reparto se realizó siguiendo dos reglas:

1. **Manipulación de bits y bytes:** todas las funciones que leen o escriben bits, calculan la cantidad de bytes o reservan y copian buffers de memoria quedan a cargo de Andrés Felipe Sierra Fernández.
2. **Equilibrio de carga:** para que el trabajo quede dividido de forma equitativa, Andrés Felipe Sierra Fernández también asume las funciones de caída y relleno del tablero, que mueven fichas escribiendo directamente sobre el buffer. Jesús David Guardiola se encarga de la interacción con el usuario, la detección de combinaciones, la lógica del juego y el `main`.

Resumen del reparto

Integrante	<code>memoriatablero.cpp</code>	<code>tablero.cpp</code>	<code>main.cpp</code>	Total
Andrés Felipe Sierra Fernández	13	3	0	16
Jesús David Guardiola	2	14	1	17

3. Responsabilidades de Andrés Felipe Sierra Fernández

Rol: núcleo de memoria y manipulación de bits y bytes (16 funciones).

3.1 Archivo `memoriatablero.cpp`

Función	Responsabilidad
calcularBytesTablero	Calcula la cantidad de bytes necesarios para el tablero mediante $(\text{filas} * \text{columnas} * 3 + 7) / 8$.
crearTablero	Reserva dinámicamente el arreglo de bytes que almacena el tablero.
inicializarBuffer	Llena el buffer con ceros para facilitar la manipulación de bits.
calcularPosicion	Convierte fila y columna en una posición lineal $(\text{fila} * \text{columnas} + \text{columna})$.
guardarFicha	Escribe los 3 bits de una ficha en el buffer, incluso cuando quedan repartidos entre dos bytes.
obtenerFicha	Lee los 3 bits de una ficha y reconstruye su valor.
imprimirBits	Muestra la representación binaria de los bytes del tablero.
hacerCaerColumna	Baja las fichas de una columna sobre los espacios vacíos (valor 6).
agregarFila	Crea un buffer nuevo, copia las fichas existentes e inserta una fila con fichas nuevas.
agregarColumna	Crea un buffer nuevo, copia las fichas existentes e inserta una columna con fichas nuevas.
eliminarFila	Redimensiona el buffer eliminando una fila.
eliminarColumna	Redimensiona el buffer eliminando una columna.
redimensionarEliminadas	Redimensiona el arreglo auxiliar de posiciones eliminadas.

3.2 Archivo tablero.cpp

Función	Responsabilidad
eliminarFichas	Marca como vacías (valor 6) las posiciones detectadas para eliminar.
hacerCaerTablero	Aplica la caída a todas las columnas del tablero.
rellenarTablero	Reemplaza los espacios vacíos con fichas nuevas generadas al azar.

4. Responsabilidades de Jesús David Guardiola

Rol: interacción con el usuario, lógica del juego y flujo principal (17 funciones).

4.1 Archivo tablero.cpp

Función	Responsabilidad
<code>pedirFilas</code>	Solicita al usuario la cantidad de filas y la valida.
<code>pedirColumnas</code>	Solicita al usuario la cantidad de columnas y la valida.
<code>filaSeleccion</code>	Pide una fila y verifica que esté dentro del rango.
<code>columnaSeleccion</code>	Pide una columna y verifica que esté dentro del rango.
<code>pedirPosicion</code>	Combina fila y columna seleccionadas en una posición lineal.
<code>pedirAccion</code>	Muestra el menú de opciones y lee la acción elegida.
<code>generarFicha</code>	Genera una ficha aleatoria con valor entre 0 y 5.
<code>asignarLetra</code>	Convierte el valor numérico de la ficha en una letra (A–F) para mostrarla.
<code>llenarTablero</code>	Llena el tablero completo con fichas aleatorias.
<code>mostrarTablero</code>	Imprime el tablero en consola usando letras.
<code>combHorizontales</code>	Cuenta fichas iguales consecutivas hacia la derecha.
<code>combVerticales</code>	Cuenta fichas iguales consecutivas hacia abajo.
<code>estaEliminada</code>	Verifica si una posición ya fue marcada para eliminar.
<code>buscarCombinaciones</code>	Detecta combinaciones de 3 o más fichas y llena la lista de posiciones a eliminar.

4.2 Archivo `memoriatablero.cpp`

Función	Responsabilidad
<code>procesarEliminacionFicha</code>	Elimina la ficha elegida por el usuario, hace caer las superiores y rellena el tablero.
<code>procesarCascadas</code>	Repite detección, eliminación, caída y relleno mientras haya combinaciones; actualiza el puntaje.

4.3 Archivo `main.cpp`

Función	Responsabilidad
<code>main</code>	Coordina el flujo del programa: pide dimensiones, crea el tablero, ejecuta el menú y muestra las estadísticas.

5. Observaciones sobre el código

Durante la revisión se detectaron los siguientes puntos que conviene corregir antes de la sustentación:

1. **`inicializarBuffer` (Andrés Felipe Sierra Fernández):** la función recibe el número de bytes, pero lo multiplica nuevamente por 3 y lo divide entre 8. Como resultado, pone en cero menos bytes de los que tiene el buffer y el resto puede quedar con basura.

2. **pedirPosicion (Jesús David Guardiola):** dentro del ciclo **while** se vuelven a declarar **int fila** e **int columna**. Esas variables nuevas solo existen dentro del ciclo, por lo que las originales nunca se actualizan y puede producirse un ciclo infinito ante una entrada inválida.
3. **asignarLetra (Jesús David Guardiola):** no retorna ningún valor cuando la ficha es 6 (posición vacía), lo que genera comportamiento indefinido.

6. Conclusión

El trabajo quedó dividido en **16 funciones** para Andrés Felipe Sierra Fernández y **17 funciones** para Jesús David Guardiola. Andrés Felipe Sierra Fernández concentra la parte técnica de bits, bytes y memoria dinámica, mientras que Jesús David Guardiola desarrolla la lógica del juego, la entrada de datos y la coordinación general en **main**, lo que permite una distribución equilibrada y coherente con la arquitectura del proyecto.